

1 - Context

🕒 Created	June 23, 2026 10:48 AM
🏷️ Tags	

1.1 Mục đích hệ thống

Hệ thống là một gateway (đóng vai trò như một bảo vệ) đứng trước các API trên cloud. Với nhiệm vụ là kiểm tra và chỉ cho những người dùng và ứng dụng đã được xác thực (authenticated) và đã được cấp quyền (authorized) để có thể gọi và sử dụng API, đồng thời kiểm tra xem yêu cầu đó là thật và integrity hay không

1.2 Actors

Actor	Mục đích	Mức tin cậy
End user	Đăng nhập, gọi API thay mình	untrusted
service, M2M machine	Gọi API tự động không qua người	semi - trusted
IdP	Cấp và ký danh tính (trust anchor)	trusted
backend services	cung cấp dịch vụ thật phía sau gateway	semi- trusted
admin	vận hành và cấu hình hệ thống	trusted
attacker	tấn công và khai thác lỗ hổng, đánh cắp thông tin người dùng	untrusted

1.3 Assets

Assets	Priority	Tại sao quý	Hậu quả nếu lộ/mất/sửa
Dữ liệu và AIP	P1	Mọi thứ bảo vệ chỉ để bảo vệ cái này, có nó là có tất cả	Mất tất cả

Assets	Priority	Tại sao quý	Hậu quả nếu lộ/mất/sửa
Access Token	P2	là vé vào API backend ngắn hạn	Hacker có thể truy cập API backend trong thời gian ngắn
Refresh Token	P1	là vé vào API backend dài hạn	Hacker có thể truy cập API backend trong thời gian dài
Symmetric key (HMAC)	P1	cùng 1 khóa để ký lẫn kiểm tra	Khả năng bị lộ khóa cao, nếu lộ thì thảm họa
Asymmetric key (ECDSA)	P1	Cần 2 khóa để ký, 1 khóa private và 1 khóa public	Khả năng bị lộ khóa thấp, nếu lộ private key thì cũng thảm họa
PII người dùng / Danh tính người dùng	P1	Danh tính người dùng cuối, có thể nói là tài sản rất quan trọng mà cần bảo vệ	Lộ quyền riêng tư mà không cần chạm vào Backend
audit log	P1	ghi lại cách hành động bất thường cũng như ghi lại dấu vết của bất kì hành động hay dấu chân nào của Hacker	Nếu mất tính toàn vẹn của log thì sẽ không bao giờ tracking được dấu vết, và thảm họa nếu log lộ rò PII

⇒ Lý do ưu tiên lưu ES256 hơn HS256 vì nó sẽ an toàn hơn nếu bị lộ

1.4 Trust boundaries

Ranh giới tin cậy = chỗ dữ liệu đi từ vùng tin cậy thấp sang cao (hoặc ngược lại). Mỗi lần băng qua = **BẮT BUỘC** có một cửa kiểm soát.

"Có ranh giới tin cậy" **KHÔNG** có nghĩa là cấm cửa. Nó nghĩa là: hai bên khác mức tin cậy → dữ liệu phải đi qua **MỘT CỬA KIỂM TRA**. Qua cửa rồi thì mới tin.

Các cặp trao đổi dữ liệu	Ranh giới tin cậy ?	Bên nào không tin bên nào	Control
Internet ↔ Gateway	có	Gateway không được tin client	Kiểm soát bằng cách sử dụng mã hóa TLS, kết hợp sử dụng JWT và HMAC để xác nhận chữ ký, rate-limit để tránh spam

Các cặp trao đổi dữ liệu	Ranh giới tin cậy ?	Bên nào không tin bên nào	Control
Gateway ↔ IdP	có	Gateway chỉ tin token sau khi kiểm chữ kí	Verify chữ kí bằng JWKS và kết hợp mã hóa TLS
Gateway ↔ Backend services	có	Backend không nên tin gateway mù quáng	Chỉ cho gateway đến backend, không đi đường vòng và backend phải tự verify không tin mù quáng
Gateway ↔ Secret store	có	Vault không đưa khóa cho ai hỏi cũng cho	gateway phải qua bước auth và mã hóa tls, cần authorize tối thiểu

1.5 Assumptions & Constraints

Assumptions

Assumptions (giả định): điều ta tin là đúng. Nếu một giả định gãy, một control sẽ ÂM THẨM hỏng mà không báo lỗi.

1. Có TSL: vì JWT và HMAC chỉ bảo vệ tính toàn vẹn và xác thực, không dấu nội dung token trên đường truyền, TLS mã hóa kênh truyền (trong đồ án chỉ mới triển khai ở http, không phải https : <http://localhost>)
2. NTP (Network Time Protocol): HMAC kiểm timestamp trong cửa sổ 300s, JWT kiểm exp, nếu đồng hồ lệch nhiều → request thật bị từ chối hoặc cửa sổ chống replay vô dụng
3. Mạng nội bộ là semi-trusted và không phơi ra Internet :
Redis/Vault/Keycloak không được cho truy cập thẳng từ ngoài

Contrains

1. Với user: Không dùng flow gửi thẳng username/password, phải dùng OAuth2 (Authorization Code + PKCE), token cấp phát phải là loại short-lived và phải có refresh token để duy trì
2. Với S2S: bắt buộc dùng luồng Client Credential, request phải được ký bằng HMAC và có mang theo Access Token
3. Phải có hệ thống cung cấp danh tính độc lập (IdP), Backend ủy thác xác thực cho gateway; nhưng phải đảm bảo backend chỉ gọi được qua gateway

(control mạng), không tin mù quáng.

4. Gateway phải là người gác cổng duy nhất, phải tự động kiểm tra token, xác minh chữ ký HMAC, kiểm tra quyền (scope), rate limiting và ghi log trước khi cho phép request đi vào backend
5. Không lưu key/password trong mã nguồn, bắt buộc dùng Vault/KMS để lưu và ký
6. Phải dùng nhiều thuật toán ký JWT khác nhau: HS256, RS256, ES256

1.6 Use-cases

Luồng A:

Người dùng cuối tương tác với app (hệ thống áp dụng Authorization code + PKCE), khi đăng nhập, người dùng bị chuyển hướng sang IdP đăng nhập, IdP sẽ trả token về cho user, lúc này user sẽ lấy token đó và gọi /api/protected (một endpoint tài nguyên) qua gateway, gateway sẽ verify chữ ký và nếu đúng thì sẽ cho qua backend

Luồng B:

M2M (hệ thống áp dụng Client Credentials + HMAC): service tự đi xin token bằng client_id/secret, sau khi có token thì sẽ lấy token và ký HMAC cái request, sau đó gọi api /api/service qua gateway, gateway sẽ verify cả token và chữ ký HMAC, áp dụng nonce để chống replay, nếu được sẽ chuyển qua backend